

robocar-unified

Single-Board AI Robot Car

Assembly, Wiring & Firmware Build Guide

A hands-on guide to assembling the consolidated robocar on a **Seeed Studio XIAO ESP32-S3 Sense**. Camera capture, Gemini AI planning, motor control, and peripherals all run on one module.

Target MCU	ESP32-S3 (8 MB PSRAM / flash)
Toolchain	ESP-IDF v5.4 (containerized)
Difficulty	Intermediate · ~2–3 h

Contents

1 · Overview	3
2 · Bill of Materials	3
Tools required	4
3 · System Architecture	4
4 · Wiring Reference	5
4.1 · XIAO ESP32-S3 GPIO assignments	5
4.2 · I ² C topology (TCA9548A @ 0x70)	5
4.3 · PCA9685 channel map (0x40, 100 Hz)	5
4.4 · Ultrasonic rangefinder	6
4.5 · Audio output (MAX98357A I ² S amplifier)	6
5 · Power	6
6 · Assembly Steps	7
7 · Build & Flash the Firmware	7
8 · First Boot & Provisioning	8
8.1 · WiFi over the USB console (Improv Serial)	8
8.2 · Discovery & AI backend	8
8.3 · Over-the-air updates	8
9 · Functional Checkout	8
10 · Troubleshooting	9

1 · Overview

The **robocar-unified** project consolidates the original dual-ESP32 robocar (a Heltec main controller plus an ESP32-CAM) onto a single **XIAO ESP32-S3 Sense**. One module now handles camera capture, AI inference, motor control, and all peripherals, using dual-core affinity to keep motor timing isolated from bursty network and vision work.

The control system is **hierarchical**: a slow **planner** (Core 1) calls Google's Gemini Robotics-ER to emit structured goals, and a fast **reactive executor** (~30 Hz, Core 0) drives the robot smoothly toward those goals while an ultrasonic sensor provides an independent obstacle reflex. The planner is not on a fixed schedule: the board boots **dormant** and asks Gemini only when something happened, backing off 15 s → 300 s when nothing does.

Core 0 — real-time

Reactive executor (visual servo, heading hold, motor PWM), peripheral I/O, command console, and the ultrasonic obstacle reflex.

Core 1 — bursty I/O

Planner (Gemini calls), OV3660 camera DMA, audio, WiFi / MQTT / OTA.

What you get

A two-wheel-drive car that captures frames, asks Gemini what to do, and drives toward goals — with pan/tilt camera, status LEDs, an OLED display, buzzer feedback, WiFi provisioning over the USB console, and over-the-air updates.

2 · Bill of Materials

Qty	Component	Notes
1	XIAO ESP32-S3 Sense	MCU + OV3660 camera + PDM mic + 8 MB PSRAM, USB-C
1	TCA9548A I ² C multiplexer	Breakout, address 0x70
1	PCA9685 16-ch PWM driver	Breakout, address 0x40
1	TB6612FNG dual motor driver	Breakout
1	SSD1306 OLED display	128×64, I ² C, address 0x3C
1	MCP23017 GPIO expander	Breakout, address 0x20 — optional
1	MAX98357A I ² S class-D amplifier	Mono, for voice output
1	Speaker	4–8 Ω, 2–3 W
1	Ultrasonic rangefinder	3.3 V variant: HC-SR04P / RCWL-1601 / US-100
2	RGB LED	Common-anode
2	SG90 micro servo	Pan / tilt
2	DC gear motor + wheel	~3–6 V hobby motors
1	Piezo buzzer	Passive
1	100 Ω resistor	In series with buzzer
1	Electrolytic capacitor	≥470 μF — for MAX98357A supply
2	18650 Li-ion cell + holder	Wired in SERIES — 7.4 V nominal
1	LM2596 buck converter module	Adjustable — set to 5.0 V
—	Chassis, wiring, headers	2WD car chassis, jumper wires, standoffs

Tools required

Soldering iron + solder, wire strippers, small screwdriver set, multimeter (for verifying 5 V rail and continuity), a USB-C cable, and a computer with Docker (for the containerized firmware build).

Sensor voltage — read this

The ultrasonic sensor **must be a 3.3 V-compatible module** (HC-SR04P, not the classic 5 V HC-SR04). The XIAO's GPIOs are not 5 V-tolerant — a 5 V ECHO line can damage the board.

3 · System Architecture

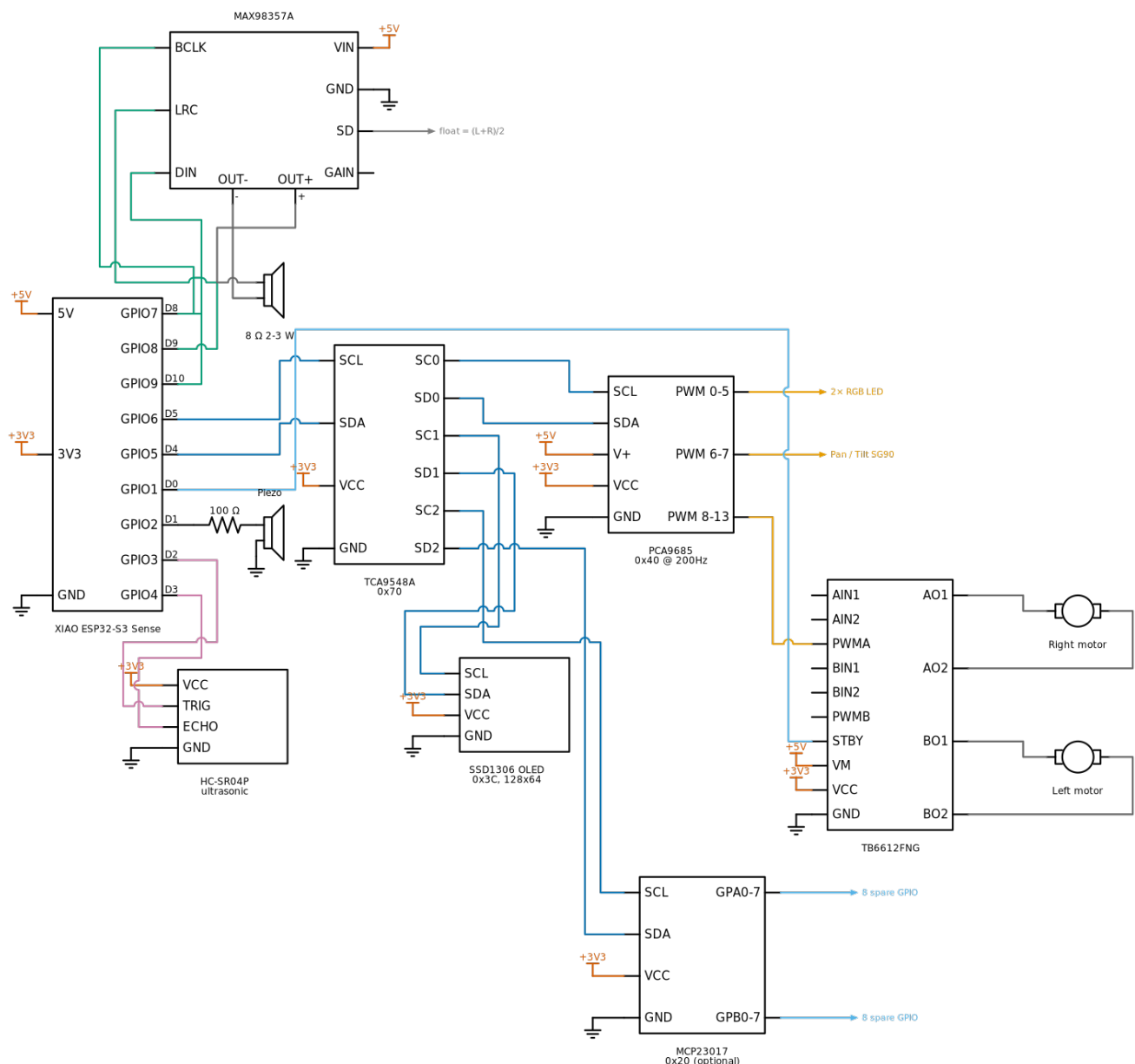


Figure 1: Full wiring schematic. Source: docs/schematics/circuits/robocar_unified.py.

Everything on the I²C bus hangs off the **TCA9548A multiplexer** — the ESP32-S3 never talks to the PCA9685 or OLED directly. This lets devices that would otherwise share addresses coexist, and keeps the two 400 kHz devices on separate channels. The ESP32-S3 itself drives only five groups of signals directly: I²C (GPIO5/6), the motor-enable STBY line (GPIO1), the buzzer (GPIO2), the ultrasonic sensor (GPIO3/4), and **I²S audio** (GPIO7/8/9) to the MAX98357A amplifier.

4 · Wiring Reference

4.1 · XIAO ESP32-S3 GPIO assignments

Only 11 GPIOs are exposed on the XIAO headers. Camera pins are internal to the Sense module and do not conflict.

XIAO Pin	GPIO	Function	Notes
D0	GPIO1	TB6612FNG STBY	HIGH = motors enabled
D1	GPIO2	Piezo buzzer	LEDC PWM · 100 Ω in series
D2	GPIO3	Ultrasonic TRIG	10 μ s pulse output
D3	GPIO4	Ultrasonic ECHO	Pulse width in (RMT RX)
D4	GPIO5	I²C SDA	to TCA9548A
D5	GPIO6	I²C SCL	to TCA9548A
D6	GPIO43	UART0 TX	Not wired · console is on USB-C
D7	GPIO44	UART0 RX	Not wired · console is on USB-C
D8	GPIO7	I²S BCLK	to MAX98357A — bit clock
D9	GPIO8	I²S LRCLK	to MAX98357A — word select
D10	GPIO9	I²S DIN	to MAX98357A — serial data

I²C runs at **400 kHz**.

GPIO budget fully allocated

There are no spare header pins. Additional digital I/O must go through the MCP23017 on TCA9548A channel 2.

4.2 · I²C topology (TCA9548A @ 0x70)

Select the channel on the multiplexer **before** addressing any downstream device. The MCP23017 on ch2 is optional — the firmware boots fine without it.

Channel	Device	Address
ch0	PCA9685 PWM driver (motors, servos, LEDs)	0x40 @ 100 Hz
ch1	SSD1306 OLED display (128×64)	0x3C
ch2	MCP23017 GPIO expander (optional)	0x20
ch3–7	<i>reserved — IMU / ToF / future sensors</i>	—

4.3 · PCA9685 channel map (0x40, 100 Hz)

All motor direction, motor PWM, servo, and LED outputs go through the PCA9685. Motor direction pins use PCA9685 “full-on” (4096) / “full-off” (0); PWM pins use the full 12-bit range (0–4095).

Ch	Signal	Ch	Signal
0	Left LED — R	8	Motor R — PWM → PWMA
1	Left LED — G	9	Motor R — IN2 (dir) → AIN2
2	Left LED — B	10	Motor R — IN1 (dir) → AIN1
3	Right LED — R	11	Motor L — IN1 (dir) → BIN1
4	Right LED — G	12	Motor L — IN2 (dir) → BIN2
5	Right LED — B	13	Motor L — PWM → PWMB
6	Pan servo (SG90)	14–15	<i>reserved</i>

Ch Signal**7 Tilt servo (SG90)**

100 Hz is a chip-wide compromise: one prescaler serves the servos, the motors and the LEDs. Measured on this build — the SG90s fitted track at 50, 100 and 125 Hz and buzz at 200, so this sits one rung below the highest rate that worked. Raise it for steadier LEDs, lower it for happier servos; `servo_freq <hz>` retunes it live, without a reflash.

4.4 · Ultrasonic rangefinder

Signal	Pin	Voltage	Function
TRIG	GPIO3 (D2)	3.3 V	10 µs pulse triggers a measurement
ECHO	GPIO4 (D3)	3.3 V	Pulse width encodes distance (RMT RX)
VCC	3.3 V	3.3 V	3.3 V variant only
GND	any GND	—	Shared ground

The sensor samples at ~20 Hz. **Obstacle reflex:** if distance < 15 cm, the executor immediately stops and reverses, independent of planner goals.

4.5 · Audio output (MAX98357A I²S amplifier)

The robot speaks through a mono I²S class-D amplifier. Audio is 24 kHz 16-bit PCM — the native output rate of the Gemini TTS model — decoded incrementally into a PSRAM ring so playback starts before the download finishes.

Signal	Pin	Function
BCLK	GPIO7 (D8)	Bit clock
LRCLK	GPIO8 (D9)	Word select / left-right clock
DIN	GPIO9 (D10)	Serial audio data to amplifier
Vin	5 V	Power — see §5 for supply requirements
GND	any GND	Shared ground
SD_MODE	floating	(L+R)/2 — firmware duplicates mono into both slots
GAIN	float	9 dB default

The I²S channel is disabled between utterances — the MAX98357A hisses faintly whenever BCLK is running, so leaving it clocking silence is audible.

Trade-off

Wiring the amplifier on D8–D10 **replaces the microSD slot**. On the Sense expansion board these three pins are the SD card's SPI bus. There is no alternative pin set — I²S needs a hardware peripheral, so it cannot be moved behind the PCA9685 or the MCP23017.

5 · Power

Power the car from a **2×18650 pack wired in series** (7.4 V nominal, 8.4 V charged) through an **LM2596 buck converter set to 5.0 V**. Series and buck go together: a step-down regulator needs its input above its output, so a parallel 3.7 V pack could not feed it. Distribute that 5 V rail to the XIAO 5 V pin, the TB6612FNG (VM + VCC), the PCA9685 (V+ and VCC), and the **MAX98357A amplifier (Vin)**; the servos take their power from the PCA9685's V+ terminal.

Golden rule

Common ground everywhere. Every module — the regulator, XIAO, motor driver, PCA9685, servos, sensors — must share one GND. Missing grounds cause brown-outs, I²C lockups, and erratic motion.

The rail dies before the cells do: the LM2596 needs its input about **1.25 V above its output at 3 A** (0.95 V at 1 A), so 5 V out drops out near **6.3 V of pack** — roughly 3.15 V per cell, which a 2S pack reaches while it still has usable charge. The rail sags rather than shutting down cleanly, and nothing announces it. Weak servos, clipping audio and random resets are all plausible low-battery symptoms; measure the pack before diagnosing anything else on this rail.

Brown-out detection is disabled in firmware because motor inrush was tripping it; a stiff 5 V supply and thick power wires matter.

The 3.3 V logic for the OLED, ultrasonic sensor, TCA9548A, and MCP23017 comes from the XIAO's 3V3 pin. Keep motor/servo current (high, noisy) on the 5 V rail and logic on 3V3.

Amplifier supply — read this: The MAX98357A draws up to 1 A peaks into a 4 Ω load. With brown-out detection already disabled for motor inrush, an undersized rail will not warn you — it will present as random resets or corrupt audio mid-sentence. Fit a **$\geq 470 \mu\text{F}$ bulk capacitor** at the amplifier's V_{in} , plus the usual 0.1 μF close to the pin, and the same at the PCA9685's $V+$ — the servos are the harsher transient source. Star-wire the rail: every load takes its own feed from the regulator's output terminal, because a servo's inrush travelling through the amplifier's feed wire is heard as distortion. Never power servos from the XIAO's 5 V pin or from USB. An **8 Ω speaker** roughly halves peak current versus 4 Ω .

6 · Assembly Steps

1. **Mount the mechanics.** Fit the two gear motors and wheels to the chassis, add the caster/third wheel, and mount the battery holder low and centered.
2. **Wire power first.** Connect the series 18650 pack to the LM2596 input and, with nothing on its output, turn the trimpot until a multimeter reads **5.0 V**. The adjustable module spans 1.2–37 V and arrives set to anything; above 5.5 V damages the amplifier and the XIAO's regulator input. Only then run the 5 V and shared GND rails.
3. **Place the XIAO** and bring out I²C (GPIO5/6), STBY (GPIO1), buzzer (GPIO2), ultrasonic pins (GPIO3/4), and I²S (GPIO7/8/9).
4. **Wire the I²C chain:** XIAO SDA/SCL → TCA9548A → PCA9685 (ch0), OLED (ch1), and optionally MCP23017 (ch2). Pull-ups on the breakouts are usually sufficient.
5. **Wire the motor driver:** PCA9685 ch8–13 → TB6612FNG inputs; STBY → GPIO1 motor outputs → the two DC motors; VM/VCC → 5 V.
6. **Add servos** (PCA9685 ch6/7) and **RGB LEDs** (ch0–5, common-anode).
7. **Add the buzzer** on GPIO2 through the 100 Ω resistor, and the ultrasonic sensor on GPIO3/4 (3.3 V power).
8. **Wire the audio path:** MAX98357A BCLK/LRC/DIN → GPIO7/8/9, V_{in} → 5 V with $\geq 470 \mu\text{F}$ bulk cap, speaker → amp output. Leave SD_MODE floating for (L+R)/2.
9. **Double-check the 3.3 V vs 5 V rails** and confirm common ground with a multimeter continuity test before first power-up.

7 · Build & Flash the Firmware

Builds run inside a container — **no local ESP-IDF install is required**. The XIAO uses native USB-Serial-JTAG (USB VID 0x303a), which just auto-detects.

```
# From the repo root
just robocar-unified::build # containerized ESP-IDF v5.4 build
PORT=/dev/cu.usbmodem* just robocar-unified::flash # flash over USB-C
just robocar-unified::monitor # serial console
# or, in one step:
just robocar-unified::flash-monitor
```

Can't enter download mode?

Hold **BOOT**, tap **RESET**, then release **BOOT** to force the bootloader, and re-run the flash command.

The flasher writes three images to an 8 MB, OTA-capable layout:

Offset	Image	Partition
0x0	build/bootloader/bootloader.bin	bootloader
0x8000	build/partition_table/partition-table.bin	partition table
0x12000	build/robocar-unified.bin	ota_0 (app)

8 · First Boot & Provisioning

8.1 · WiFi over the USB console (Improv Serial)

No WiFi credentials are compiled in. On first boot the device speaks **Improv Serial** on the same USB port you flashed it with – not the BLE variant, so there is nothing to pair. Open the board in a browser-based Improv Serial provisioner (Chrome desktop, e.g. ESP Web Tools) and send your SSID and password. They are written to NVS only after they are proven to connect, so a typo cannot overwrite a working network.

For local development you can instead copy `main/credentials.h.example` to `main/credentials.h` (gitignored) and hard-code credentials.

8.2 · Discovery & AI backend

After connecting, the car is reachable at **robocar-unified.local** via mDNS. The planner uses **Gemini Robotics-ER 2** to emit goals – `drive()`, `track()`, `rotate()`, and `stop()` – plus `speak()`, which renders a sentence through a second TTS model and plays it while the robot keeps driving.

8.3 · Over-the-air updates

OTA is enabled with app rollback. The updater pulls releases from the `laurigates/mcu-tinkering-lab` GitHub repo; `version.txt` is the single source of truth for the running version and is managed by `release-please`.

9 · Functional Checkout

Work through these after first flash, watching the serial monitor:

✓	Check	Expected result
☐	Boot log	No PSRAM / boot-loop errors; tasks pin to cores 0 & 1
☐	I ² C scan	TCA9548A, PCA9685, and OLED all detected
☐	OLED	Status screen renders (128×64)
☐	LEDs	Both RGB LEDs cycle / show status colors
☐	Buzzer	Audible startup tone

✓	Check	Expected result
☐	Servos	Pan/tilt center, then sweep within limits
☐	Motors	Both wheels drive forward and reverse; STBY HIGH
☐	Ultrasonic	Distance readings track a hand moving closer/away
☐	Reflex	Car stops/reverses when an obstacle is < 15 cm
☐	Audio / TTS	Robot speaks startup message; speech queues without blocking motion
☐	Microphone	mic reports frames arriving, not gate: DEAF; listen answers a spoken question
☐	WiFi	Provisions via Improv; robocar-unified.local resolves

10 · Troubleshooting

Symptom	Likely cause & fix
Boot loop on power-up	Wrong PSRAM mode. The Sense uses octal PSRAM (CONFIG_SPIRAM_MODE_OCT=y); don't change it.
Random resets under motor load	Weak 5 V rail / missing common ground. Use thicker power wires and verify the LM2596 holds 5 V under load — scope it, a multimeter samples too slowly to see a millisecond sag.
Servos buzz but do not move	Almost always supply, not signal. Check V+ on the PCA9685 is fed from the regulator (VCC powers only the logic), and that the pack is above 6.3 V. <code>servo exercise</code> on the console logs every write, so a still servo with successful writes is a power fault.
No I ² C devices found	Not selecting the TCA9548A channel first, or SDA/SCL swapped. Check GPIO5=SDA, GPIO6=SCL.
OLED and PCA9685 conflict	Both bypassing the mux. Route each through its own TCA9548A channel (ch1 / ch0).
Motors don't move	STBY (GPIO1) not HIGH, or VM not on 5 V. Confirm TB6612FNG power and enable line.
Servos jitter or buzz	Shared noisy rail: keep servo power on 5 V with common ground. A servo that BUZZES and holds against a stop is the PWM frame rate, not the rail — these SG90s track at 50–125 Hz and stall at 200. <code>servo freq <hz></code> retunes it live.
Board won't flash	Force download mode: hold BOOT, tap RESET, release BOOT.
Damaged ECHO / no distance	Used a 5 V HC-SR04. Replace with a 3.3 V module (HC-SR04P).
No audio / distorted speech	Weak MAX98357A supply. Fit ≥470 µF bulk cap at Vin, or use a separate 5 V feed. Check I2S wiring on GPIO7 – 9.

Authoritative pin data: `packages/robocar/unified/main/pin_config.h` · Design rationale: `docs/decisions/ADR-016-hierarchical-ai-controller.md`

Regenerate this PDF with `typst compile --root ../../../../ build-guide.typ`.